Paystack, Deposit, and Profit Review Findings

Date: April 22, 2026

Review type: Static code review only

Findings

1. High: customer deposit credit is derived from browser-supplied metadata instead of strict server-side calculation.

The backend reads `metadata.fee_rate` and `metadata.net_amount_ghs`, and only recomputes if those values are missing. The browser sends both fields, so a tampered client can reduce the fee or change the net credited amount up to the gross paid amount.

References:

- `deposit.py:132`
- `deposit.py:134`
- `deposit.py:140`
- `templates/deposit.html:267`
- `templates/deposit.html:268`
- `templates/deposit.html:269`
- `templates/admin_wallet.html:335`
- `templates/admin_wallet.html:336`
- `templates/admin_wallet.html:337`

2. High: one customer deposit is being represented in two admin balances at the same time.

The customer deposit flow credits the admin wallet directly, then also credits the admin Paystack payout ledger as available balance. That creates a double-spend risk because the same money can be used inside the app and also withdrawn from the payout ledger.

References:

- `deposit.py:153`
- `deposit.py:155`
- `deposit.py:191`
- `admin_paystack_ledger.py:133`
- `admin_paystack_ledger.py:136`
- `admin_paystack_ledger.py:171`
- `admin_paystack_ledger.py:172`

3. High: store profit is credited immediately on paid checkout even when fulfillment fails, and I did not find a matching reversal in the refund path.

The Bulk SMS branch can mark a line as failed but still assigns `store_profit_amount`, and order finalization sums all `store_profit_amount` values into the store profit balance without filtering by line outcome. The refund flow credits the buyer wallet, but does not reverse store profit or the Paystack ledger there.

References:

- `routes/store_page.py:3177`
- `routes/store_page.py:3180`
- `routes/store_page.py:3203`
- `routes/store_page.py:4240`
- `routes/store_page.py:4300`
- `admin_orders.py:1181`
- `admin_orders.py:1198`

4. High: store Paystack inflow is credited using the actual paid amount, not the repriced expected total.

Underpayment is blocked, but overpayment is accepted, written into the transaction as `paid_ghs`, and also credited into the admin Paystack ledger. That inflates payoutable balance instead of isolating the difference for reconciliation.

References:

- `routes/store_page.py:2706`
- `routes/store_page.py:2742`
- `routes/store_page.py:2768`
- `routes/store_page.py:2785`
- `routes/store_page.py:2802`
- `routes/store_page.py:2804`

5. Medium: `profit_amount_total` does not include AFA, results checker, or Bulk SMS lines even though those lines do have store-side profit.

In the special branches, the code records `store_profit_amount` but hard-codes `profit_amount` to `0.0`, while only the generic and social branches add to `profit_amount_total`. That understates order-level profit reporting for those services.

References:

- `routes/store_page.py:3016`
- `routes/store_page.py:3026`
- `routes/store_page.py:3101`
- `routes/store_page.py:3111`
- `routes/store_page.py:3187`
- `routes/store_page.py:3203`
- `routes/store_page.py:3293`

- `routes/store_page.py:3410`

6. Medium: customer deposit ownership is recorded inconsistently.

The money is credited to the admin wallet, but the transaction row is saved under the customer `user_id`. That makes customer deposit reporting misleading and pollutes customer KPIs that sum deposits by `user_id`.

References:

- `deposit.py:153`
- `deposit.py:167`
- `deposit.py:184`
- `transactions.py:73`
- `transactions.py:74`
- `transactions.py:125`

7. Medium: customer-origin deposits do not write a balance log, but admin self-deposits do.

One money-in path therefore has a weaker audit trail than the other.

References:

- `deposit.py:155`
- `deposit.py:165`
- `deposit.py:380`
- `deposit.py:391`
- `deposit.py:407`

8. Medium: the Paystack and deposit idempotency pattern is race-prone.

The code repeatedly uses `find_one(...)` and then `insert_one(...)` for deposits, store Paystack transactions, and admin Paystack balance credits, but I did not find unique-index enforcement for those reference or dedupe fields in the reviewed files. Parallel retries or duplicate callbacks can still double-credit.

References:

- `deposit.py:128`
- `deposit.py:165`
- `deposit.py:352`
- `deposit.py:407`
- `routes/store_page.py:2793`
- `admin_paystack_ledger.py:124`
- `admin_paystack_ledger.py:126`
- `admin_paystack_ledger.py:142`

9. Low: non-main-admins are forced onto the `store` Paystack profile view.

They cannot inspect deposit-profile records from the audit page even though deposit flows affect their balances, which weakens reconciliation for normal admins.

References:

- `paystack_transactions.py:74`
- `paystack_transactions.py:77`
- `paystack_transactions.py:78`

Notes

The main accounting weakness is that the app is mixing three different balance concepts without one strict source of truth.

- Wallet balance
- Paystack payout balance
- Profit tracking

That is why the strongest risks are:

- double-counted deposits
- overstated payoutable Paystack inflow
- understated or mistimed service profit

Scope

This was a static code review only.

- I did not run live Paystack payments.
- I did not execute payout requests.
- I did not run live refund scenarios in the browser.